TASK 5: PAYMENT TRANSACTION MANAGEMENT SYSTEM

StudentDetails:

- **Student Name:** Divya. V
- **Roll Number:** E25AI111
- **Department:** B.Sc Computer Science With AI
- **Project Title:** THE ARTISAN CRUMB: Bakery & Pastry Management System

Scenario:

"THE ARTISAN CRUMB" bakery enterprise needs to maintain payment information for customer orders processed through both physical counters and the online ordering portal. The system should record payment details, track transaction status, manage successful and failed payments, and generate payment-related business reports.
Students will create a Payment Transaction Module connected directly with the existing Customer and Order Management modules created in Tasks 1 through 4.

Tasks to be Done:

1. **Create Payment Table** (Relational table definition with Primary Key and Foreign Key constraints referencing Orders).
2. **Store Payment Details** (Execute DML insertions to record multi-channel bakery settlements including retries).
3. **Manage Successful and Failed Transactions** (Audit completed versus failed settlement attempts).
4. **Analyze Payment Methods Used by Customers** (Generate executive channel analytics and revenue distributions).
5. **Customer Payment History & Payment Analysis** (End-to-end multi-table reconciliation join reports).

1. Create Payment Table

Payment Entity Data Dictionary

Stores customer payment transaction details. Attribute	Description
Payment_ID	Unique payment identifier formatted as PAY-xxx
Order_ID	Order reference linking to parent record in Orders(Order_ID)
Payment_Date	Date of transaction settlement attempt
Payment_Mode	Payment channel: UPI, Credit Card, Debit Card, Net Banking, COD
Payment_Status	Settlement status: Completed, Failed, Pending
Transaction_Amount	Paid amount in INR (NUMBER(10,2))

Create Payment Master Table

```
SQL> CREATE TABLE Payment (
  2      Payment_ID         VARCHAR2(7),
  3      OrderID            NUMBER,
  4      Payment_Date       DATE DEFAULT SYSDATE,
  5      Payment_Mode       VARCHAR2(15),
  6      Payment_Status     VARCHAR2(12),
  7      Transaction_Amount NUMBER(10,2),
  8      PRIMARY KEY (Payment_ID),
  9      FOREIGN KEY (OrderID) REFERENCES Orders (ORDERID)
 10  );

Table created.
```

Domain Integrity CHECK Constraints

```
SQL> ALTER TABLE Payment
  2      ADD CONSTRAINT chk_payment_mode
  3      CHECK (Payment_Mode IN ('UPI', 'Credit Card', 'Debit Card', 'Net Banking', 'COD'));

Table altered.

SQL> ALTER TABLE Payment
  2      ADD CONSTRAINT chk_payment_status
  3      CHECK (Payment_Status IN ('Completed', 'Failed', 'Pending'));

Table altered.

SQL> ALTER TABLE Payment
  2      ADD CONSTRAINT chk_trans_amount
  3      CHECK (Transaction_Amount > 0);

Table altered.
```

Mandatory NOT NULL Constraints

```
SQL> ALTER TABLE Payment MODIFY Payment_Date NOT NULL;

Table altered.

SQL> ALTER TABLE Payment MODIFY Payment_Mode NOT NULL;

Table altered.

SQL> ALTER TABLE Payment MODIFY Payment_Status NOT NULL;

Table altered.

SQL> ALTER TABLE Payment MODIFY Transaction_Amount NOT NULL;

Table altered.

SQL> DESC Payment;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------
----
 PAYMENT_ID                                NOT NULL VARCHAR2(7)
 ORDERID                                            NUMBER
 PAYMENT_DATE                              NOT NULL DATE
 PAYMENT_MODE                              NOT NULL VARCHAR2(15)
 PAYMENT_STATUS                            NOT NULL VARCHAR2(12)
 TRANSACTION_AMOUNT                        NOT NULL NUMBER(10,2)
```

2. Store Payment Details

Synchronized Bakery Order Headers

To support a comprehensive payment audit log and realistic volume reporting across all our registered bakery customers (CUST-101 through CUST-104), the following supplementary order records are registered alongside initial orders ORD-501 to ORD-504:

Supplementary Bakery Orders to support payment channel distribution

```
SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status)
  2  VALUES ('ORD-505', 'CUST-104', TO_DATE('2026-03-24', 'YYYY-MM-DD'), 1250.00, 'Delivered');

1 row created.

SQL>
SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status)
  2  VALUES ('ORD-506', 'CUST-102', TO_DATE('2026-03-25', 'YYYY-MM-DD'), 720.00, 'Delivered');

1 row created.

SQL>
SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status)
  2  VALUES ('ORD-507', 'CUST-103', TO_DATE('2026-03-26', 'YYYY-MM-DD'), 1450.00, 'Baking');

1 row created.

SQL>
SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status)
  2  VALUES ('ORD-508', 'CUST-101', TO_DATE('2026-03-27', 'YYYY-MM-DD'), 890.00, 'Pending');

1 row created.
```

Direct Payment Transaction Insertions

```
SQL> INSERT INTO Payment (Payment_ID, Order_ID, Payment_Date, Payment_Mode, Payment_Status, Transaction_Amount)
  2  VALUES ('PAY-101', 'ORD-501', TO_DATE('2026-03-18', 'YYYY-MM-DD'), 'Credit Card', 'Completed', 660.00);

1 row created.

SQL> INSERT INTO Payment (Payment_ID, Order_ID, Payment_Date, Payment_Mode, Payment_Status, Transaction_Amount)
  2  VALUES ('PAY-102', 'ORD-502', TO_DATE('2026-03-19', 'YYYY-MM-DD'), 'UPI', 'Completed', 850.00);

1 row created.

SQL> INSERT INTO Payment (Payment_ID, Order_ID, Payment_Date, Payment_Mode, Payment_Status, Transaction_Amount)
  2  VALUES ('PAY-103', 'ORD-503', TO_DATE('2026-03-20', 'YYYY-MM-DD'), 'Debit Card', 'Completed', 360.00);

1 row created.

SQL> INSERT INTO Payment (Payment_ID, Order_ID, Payment_Date, Payment_Mode, Payment_Status, Transaction_Amount)
  2  VALUES ('PAY-104', 'ORD-504', TO_DATE('2026-03-22', 'YYYY-MM-DD'), 'Net Banking', 'Completed', 510.00);

1 row created.

SQL>
SQL> INSERT INTO Payment (Payment_ID, Order_ID, Payment_Date, Payment_Mode, Payment_Status, Transaction_Amount)
  2  VALUES ('PAY-105', 'ORD-505', TO_DATE('2026-03-24', 'YYYY-MM-DD'), 'UPI', 'Completed', 1250.00);

1 row created.

SQL> INSERT INTO Payment (Payment_ID, Order_ID, Payment_Date, Payment_Mode, Payment_Status, Transaction_Amount)
  2  VALUES ('PAY-106', 'ORD-506', TO_DATE('2026-03-25', 'YYYY-MM-DD'), 'COD', 'Completed', 720.00);

1 row created.
```

```
SQL> INSERT INTO Payment (Payment_ID, Order_ID, Payment_Date, Payment_Mode, Payment_Status, Transaction_Amount)
  2  VALUES ('PAY-107', 'ORD-507', TO_DATE('2026-03-26', 'YYYY-MM-DD'), 'Debit Card', 'Failed', 1450.00);

1 row created.

SQL> INSERT INTO Payment (Payment_ID, Order_ID, Payment_Date, Payment_Mode, Payment_Status, Transaction_Amount)
  2  VALUES ('PAY-108', 'ORD-507', TO_DATE('2026-03-26', 'YYYY-MM-DD'), 'Credit Card', 'Completed', 1450.00);

1 row created.

SQL> INSERT INTO Payment (Payment_ID, Order_ID, Payment_Date, Payment_Mode, Payment_Status, Transaction_Amount)
  2  VALUES ('PAY-109', 'ORD-508', TO_DATE('2026-03-27', 'YYYY-MM-DD'), 'Debit Card', 'Completed', 890.00);

1 row created.
```

Verify All Recorded Payment Transactions

```
PAYMENT_ID           ORDER_ID             PAYMENT_DA
-------------------- -------------------- ----------
PAYMENT_MODE
--------------------------------------------------
PAYMENT_STATUS                                     TRANSACTION_AMOUNT
-------------------------------------------------- ------------------
PAY-101              ORD-501              2026-03-18
Credit Card
Completed                                                         660

PAY-102              ORD-502              2026-03-19
UPI
Completed                                                         850

PAYMENT_ID           ORDER_ID             PAYMENT_DA
-------------------- -------------------- ----------
PAYMENT_MODE
--------------------------------------------------
PAYMENT_STATUS                                     TRANSACTION_AMOUNT
-------------------------------------------------- ------------------

PAY-103              ORD-503              2026-03-20
Debit Card
Completed                                                         360

PAY-104              ORD-504              2026-03-22
Net Banking

PAYMENT_ID           ORDER_ID             PAYMENT_DA
-------------------- -------------------- ----------
PAYMENT_MODE
--------------------------------------------------
PAYMENT_STATUS                                     TRANSACTION_AMOUNT
-------------------------------------------------- ------------------
Completed                                                         510

PAY-105              ORD-505              2026-03-24
UPI
Completed                                                        1250

PAY-106              ORD-506              2026-03-25
```

```
PAYMENT_ID           ORDER_ID             PAYMENT_DA
-------------------- -------------------- ----------
PAYMENT_MODE
--------------------------------------------------
PAYMENT_STATUS                                     TRANSACTION_AMOUNT
-------------------------------------------------- ------------------
COD
Completed                                                         720

PAY-107              ORD-507              2026-03-26
Debit Card
Failed                                                           1450


PAYMENT_ID           ORDER_ID             PAYMENT_DA
-------------------- -------------------- ----------
PAYMENT_MODE
--------------------------------------------------
PAYMENT_STATUS                                     TRANSACTION_AMOUNT
-------------------------------------------------- ------------------
PAY-108              ORD-507              2026-03-26
Credit Card
Completed                                                        1450

PAY-109              ORD-508              2026-03-27
Debit Card
Completed                                                         890

PAYMENT_ID           ORDER_ID             PAYMENT_DA
-------------------- -------------------- ----------
PAYMENT_MODE
--------------------------------------------------
PAYMENT_STATUS                                     TRANSACTION_AMOUNT
-------------------------------------------------- ------------------


9 rows selected.
```

3. Manage Successful and Failed Transactions

Tasks:

Write SQL queries to:

1. Display all successful payments.
2. Find failed transactions.
3. Count total successful and failed payments.

1. Display all successful payments:

```
SQL> SELECT Payment_ID, Order_ID, TO_CHAR(Payment_Date, 'YYYY-MM-DD') AS Payment_Date,
  2         Payment_Mode, Payment_Status, Transaction_Amount
  3  FROM Payment
  4  WHERE Payment_Status = 'Completed'
  5  ORDER BY Payment_Date;

PAYMENT_ID           ORDER_ID             PAYMENT_DA
-------------------- -------------------- ----------
PAYMENT_MODE
--------------------------------------------------
PAYMENT_STATUS                                     TRANSACTION_AMOUNT
-------------------------------------------------- ------------------
PAY-101              ORD-501              2026-03-18
Credit Card
Completed                                                         660

PAY-102              ORD-502              2026-03-19
UPI
Completed                                                         850

PAYMENT_ID           ORDER_ID             PAYMENT_DA
-------------------- -------------------- ----------
PAYMENT_MODE
--------------------------------------------------
PAYMENT_STATUS                                     TRANSACTION_AMOUNT
-------------------------------------------------- ------------------

PAY-103              ORD-503              2026-03-20
Debit Card
Completed                                                         360

PAY-104              ORD-504              2026-03-22
Net Banking

PAYMENT_ID           ORDER_ID             PAYMENT_DA
-------------------- -------------------- ----------
PAYMENT_MODE
--------------------------------------------------
PAYMENT_STATUS                                     TRANSACTION_AMOUNT
-------------------------------------------------- ------------------
Completed                                                         510

PAY-105              ORD-505              2026-03-24
UPI
Completed                                                        1250
```

```
PAY-106              ORD-506              2026-03-25

PAYMENT_ID           ORDER_ID             PAYMENT_DA
-------------------- -------------------- ----------
PAYMENT_MODE
--------------------------------------------------
PAYMENT_STATUS                                     TRANSACTION_AMOUNT
-------------------------------------------------- ------------------
COD
Completed                                                         720

PAY-108              ORD-507              2026-03-26
Credit Card
Completed                                                        1450


PAYMENT_ID           ORDER_ID             PAYMENT_DA
-------------------- -------------------- ----------
PAYMENT_MODE
--------------------------------------------------
PAYMENT_STATUS                                     TRANSACTION_AMOUNT
-------------------------------------------------- ------------------
PAY-109              ORD-508              2026-03-27
Debit Card
Completed                                                         890


8 rows selected.
```

2. Find failed transactions:

```
SQL> SELECT Payment_ID, Order_ID, TO_CHAR(Payment_Date, 'YYYY-MM-DD') AS Payment_Date,
  2         Payment_Mode, Payment_Status, Transaction_Amount
  3  FROM Payment
  4  WHERE Payment_Status = 'Failed';

PAYMENT_ID           ORDER_ID             PAYMENT_DA
-------------------- -------------------- ----------
PAYMENT_MODE
--------------------------------------------------
PAYMENT_STATUS                                TRANSACTION_AMOUNT
--------------------------------------------- ------------------
PAY-107              ORD-507              2026-03-26
Debit Card
Failed                                                      1450
```

3. Count total successful and failed payments:

```
SQL> SELECT Payment_Status, COUNT(Payment_Status) AS Total_Count
  2  FROM Payment
  3  GROUP BY Payment_Status;

PAYMENT_STATUS                                  TOTAL_COUNT
---------------------------------------------- -----------
Failed                                                   1
Completed                                                8
```

4. Analyze Payment Methods Used by Customers

Generate payment analysis reports.

Report 1: Payment Mode Analysis

Find:

- Number of transactions using UPI.
- Number of card payments (Credit Card & Debit Card).
- Preference breakdown across all payment modes.
- Most preferred payment method.

```
SQL> SELECT COUNT(*) AS UPI_Transactions
  2  FROM Payment
  3  WHERE Payment_Mode = 'UPI';

UPI_TRANSACTIONS
----------------
               2

SQL> SELECT COUNT(*) AS Card_Payments
  2  FROM Payment
  3  WHERE Payment_Mode IN ('Credit Card', 'Debit Card');

CARD_PAYMENTS
-------------
            5

SQL> SELECT Payment_Mode, COUNT(*) AS Transaction_Count
  2  FROM Payment
  3  GROUP BY Payment_Mode
  4  ORDER BY Transaction_Count DESC;

PAYMENT_MODE                                       TRANSACTION_COUNT
-------------------------------------------------- -----------------
Debit Card                                                         3
Credit Card                                                        2
UPI                                                                2
Net Banking                                                        1
COD                                                                1
```

```
SQL> SELECT * FROM (
  2      SELECT Payment_Mode, COUNT(*) AS Transaction_Count
  3      FROM Payment
  4      GROUP BY Payment_Mode
  5      ORDER BY Transaction_Count DESC
  6  )
  7  WHERE ROWNUM = 1;

PAYMENT_MODE                                       TRANSACTION_COUNT
-------------------------------------------------- -----------------
Debit Card                                                         3
```

Report 2: Revenue Analysis

Find:

- Total revenue generated from settled orders.
- Revenue by payment method.
- Average transaction amount across completed payments.

```
SQL> SELECT SUM(Transaction_Amount) AS Total_Revenue
  2  FROM Payment
  3  WHERE Payment_Status = 'Completed';

TOTAL_REVENUE
-------------
         6690

SQL> SELECT Payment_Mode, SUM(Transaction_Amount) AS Total_Revenue
  2  FROM Payment
  3  WHERE Payment_Status = 'Completed'
  4  GROUP BY Payment_Mode
  5  ORDER BY Total_Revenue DESC;

PAYMENT_MODE                                       TOTAL_REVENUE
-------------------------------------------------- -------------
Credit Card                                                 2110
UPI                                                         2100
Debit Card                                                  1250
COD                                                          720
Net Banking                                                  510

SQL> SELECT ROUND(AVG(Transaction_Amount), 2) AS Average_Transaction_Amount
  2  FROM Payment
  3  WHERE Payment_Status = 'Completed';

AVERAGE_TRANSACTION_AMOUNT
--------------------------
                    836.25
```

5. Customer Payment History & Payment Analysis

Report 3: Customer Payment History

Display:

- Customer name (concatenated full name)
- Order ID
- Order Date
- Payment Date
- Payment Mode
- Amount
- Payment Status

```
SQL> SELECT c.first_name || ' ' || c.last_name AS Customer_Name,
  2         o.Order_ID,
  3         TO_CHAR(o.Order_Date, 'YYYY-MM-DD') AS Order_Date,
  4         TO_CHAR(p.Payment_Date, 'YYYY-MM-DD') AS Payment_Date,
  5         p.Payment_Mode,
  6         p.Transaction_Amount AS Amount,
  7         p.Payment_Status
  8  FROM customer c
  9  JOIN Orders o ON c.customer_id = o.Customer_ID
 10  JOIN Payment p ON o.Order_ID = p.Order_ID
 11  ORDER BY p.Payment_Date ASC, o.Order_ID ASC;

CUSTOMER_NAME                             ORDER_ID             ORDER_DATE
----------------------------------------- -------------------- ----------
PAYMENT_DA PAYMENT_MODE                                           AMOUNT
---------- ---------------------------------------------------- ----------
PAYMENT_STATUS
----------------------------------------------
Meera Iyer                                ORD-505              2026-03-24
2026-03-24 UPI                                                      1250
Completed

Kavya Patel                               ORD-506              2026-03-25
2026-03-25 COD                                                       720
Completed

CUSTOMER_NAME                             ORDER_ID             ORDER_DATE
----------------------------------------- -------------------- ----------
PAYMENT_DA PAYMENT_MODE                                           AMOUNT
---------- ---------------------------------------------------- ----------
PAYMENT_STATUS
----------------------------------------------

Arjun Das                                 ORD-507              2026-03-26
2026-03-26 Debit Card                                               1450
Failed

Arjun Das                                 ORD-507              2026-03-26
2026-03-26 Credit Card                                              1450

CUSTOMER_NAME                             ORDER_ID             ORDER_DATE
----------------------------------------- -------------------- ----------
PAYMENT_DA PAYMENT_MODE                                           AMOUNT
---------- ---------------------------------------------------- ----------
PAYMENT_STATUS
----------------------------------------------
Completed

Siddharth Sharma                          ORD-508              2026-03-27
2026-03-27 Debit Card                                                890
Completed
```

Comprehensive Payment Analysis Matrix

A consolidated cross-channel executive summary detailing transaction volume, realized gross revenue, and average ticket size:



PAYMENT_MODE	TRANSACTION_COUNT	TOTAL_REVENUE	AVG_TRANSACTION
Credit Card	2	2110	1055
UPI	2	2100	1050
Debit Card	2	1250	625

PAYMENT_MODE	TRANSACTION_COUNT	TOTAL_REVENUE	AVG_TRANSACTION
COD	1	720	720
Net Banking	1	510	510